JavaScript For Of

‹ Previous Next ›

The For Of Loop

The JavaScript `for of` statement loops through the values of an iterable object.

It lets you loop over iterable data structures such as Arrays, Strings, Maps, NodeLists, and more:

Syntax

```
for (variable of iterable) {
  // code block to be executed
}
```

variable - For every iteration the value of the next property is assigned to the variable. *Variable* can be declared with `const`, `let`, or `var`.

iterable - An object that has iterable properties.

Browser Support

For/of was added to JavaScript in 2015 (ES6)

Chrome 38	Edge 12	Firefox 51	Safari 7	Opera 25
Oct 2014	Jul 2015	Oct 2016	Oct 2013	Oct 2014

For/of is not supported in Internet Explorer.

Looping over an Array

Example

```
const cars = ["BMW", "Volvo", "Mini"];

let text = "";
for (let x of cars) {
  text += x;
}
```

Try it Yourself »

Looping over a String

Example

```
let language = "JavaScript";

let text = "";
for (let x of language) {
```

JavaScript parseFloat()

‹ Previous JavaScript Global Methods Next ›

Examples

Parse different values:

```
parseFloat(10);
parseFloat("10");
parseFloat("10.33");
parseFloat("34 45 66");
parseFloat("He was 40");
```

Try it Yourself »

More examples below.

Description

The `parseFloat()` method parses a value as a string and returns the first number.

Notes

JavaScript parseInt()

‹ Previous JavaScript Global Methods Next ›

Example

Parse different values:

```
parseInt("10");
parseInt("10.00");
parseInt("10.33");
parseInt("34 45 66");
parseInt(" 60 ");
parseInt("40 years");
parseInt("He was 40");
```

Try it Yourself »

Description

The `parseInt` method parses a value as a string and returns the first integer.

A radix parameter specifies the number system to use:

2 = binary, 8 = octal, 10 = decimal, 16 = hexadecimal.

JavaScript Number toFixed()

‹ Previous JavaScript Number Reference Next ›

Examples

```
let num = 5.56789;
let n = num.toFixed();
```

Try it Yourself »

```
let num = 5.56789;
let n = num.toFixed(2);
```

Try it Yourself »

More examples below

Description

The `toFixed()` method converts a number to a string.

The `toFixed()` method rounds the string to a specified number of decimals.

If the number of decimals are higher than in the number, zeros are added.

Syntax

```
number.toFixed(x)
```

Parameters

Parameter	Description
x	Optional. Number of decimals. Default is 0 (no decimals)

Return Value

Type	Description
A string	The representation of a number with (or without) decimals.

More Examples

Round to 10 decimals

```
let num = 5.56789;
let n = num.toFixed(10);
```

JavaScript Object Properties

Accessing JavaScript Properties

The syntax for accessing the property of an object is:

```
// objectName.property
let age = person.age;
```

or

```
//objectName["property"]
let age = person["age"];
```

or

```
//objectName[expression]
let age = person[x];
```

Examples

```
person.firstname + " is " + person.age + " years old.";
```

Try it Yourself »

```
person["firstname"] + " is " + person["age"] + " years old.";
```

Try it Yourself »

Examples of using fs.readFileSync in Node.js

Note: this page has been created with the use of AI. Please take caution, and note that the content of this page does not necessarily reflect the opinion of Cratecode.

The `fs.readFileSync` method is a part of the `fs` (file system) module in Node.js. It allows you to read the entire content of a file synchronously, meaning that the function will block the execution of the rest of the code until the file is completely read. This can be useful in cases where you need to read a file before proceeding with other operations in your code.

In this article, we will go through examples of using `fs.readFileSync` in Node.js.

Basic Usage

First, you need to import the `fs` module:

```
const fs = require("fs");
```

Next, use the `fs.readFileSync` method to read a file:

```
const data = fs.readFileSync("example.txt", "utf-8");
console.log(data);
```

In this example, we read the content of the file `example.txt` and store it in the `data` variable. The second argument, `"utf-8"`, specifies the encoding used to read the file. If you don't provide an encoding, the method will return a `Buffer` object instead of a string.

Handling Errors

When using `fs.readFileSync`, it's important to handle any errors that may occur, such as the file not being found. You can use a try-catch block to catch any errors:

```
const fs = require("fs");

try {
    const data = fs.readFileSync("nonexistent.txt", "utf-8");
    console.log(data);
} catch (error) {
    console.error("Error reading file:", error);
}
```

In this example, we try to read a non-existent file, and the catch block handles the error by logging it to the console.

Reading a JSON File

You can use `fs.readFileSync` to read a JSON file and parse its content:

```
const fs = require("fs");

try {
    const data = fs.readFileSync("example.json", "utf-8");
    const jsonData = JSON.parse(data);
    console.log(jsonData);
} catch (error) {
    console.error("Error reading JSON file:", error);
}
```

In this example, we read a file named `example.json`, and then parse its content using `JSON.parse()`.

JSON Syntax

‹ Previous Next ›

The JSON syntax is a subset of the JavaScript syntax.

JSON Syntax Rules

JSON syntax is derived from JavaScript object notation syntax:

- Data is in name/value pairs
- Data is separated by commas
- Curly braces hold objects
- Square brackets hold arrays

JSON Data - A Name and a Value

JSON data is written as name/value pairs (aka key/value pairs).

A name/value pair consists of a field name (in double quotes), followed by a colon, followed by a value:

Example

```
"name":"John"
```

JSON - Evaluates to JavaScript Objects

The JSON format is almost identical to JavaScript objects.

In JSON, *keys* must be strings, written with double quotes:

JSON

```
{"name":"John"}
```

In JavaScript, keys can be strings, numbers, or identifier names:

JavaScript

```
{name:"John"}
```

JSON Values

In **JSON**, *values* must be one of the following data types:

- a string
- a number
- an object
- an array
- a boolean
- null

- a date
- undefined

In JSON, *string values* must be written with double quotes:

JSON

```
{"name":"John"}
```

In JavaScript, you can write string values with double *or* single quotes:

JavaScript

```
{name:'John'}
```

JavaScript Objects

Because JSON syntax is derived from JavaScript object notation, very little extra software is needed to work with JSON within JavaScript.

With JavaScript you can create an object and assign data to it, like this:

Example

```
person = {name:"John", age:31, city:"New York"};
```

You can access a JavaScript object like this:

```
// returns John
person.name;
```

Try it Yourself »

It can also be accessed like this:

Example

```
// returns John
person["name"];
```

Try it Yourself »

Data can be modified like this:

Example

```
person.name = "Gilbert";
```

Try it Yourself »

It can also be modified like this:

Example

```
person["name"] = "Gilbert";
```

You will learn how to convert JavaScript objects into JSON later in this tutorial.

JavaScript Arrays as JSON

The same way JavaScript objects can be written as JSON, JavaScript arrays can also be written as JSON.

You will learn more about objects and arrays later in this tutorial.

JSON Files

- The file type for JSON files is ".json"
- The MIME type for JSON text is "application/json"

‹ Previous Next ›

Track your progress - it's free! Sign Up Log in